

Ejercicio 5: Subsecuencia Común de Máxima Longitud (SCML)

Enunciado

Dadas dos secuencias $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, diseñar un algoritmo de programación dinámica que encuentre la subsecuencia común de ambas que tenga longitud máxima.

1. Estructura de Programación Dinámica

Definición de la Función Óptima

Definimos una matriz DP de dimensiones $(m + 1) \times (n + 1)$, donde:

- $DP[i][j]$ es la **longitud** de la subsecuencia común de máxima longitud entre el prefijo $X_{1..i}$ (de longitud i) y el prefijo $Y_{1..j}$ (de longitud j).

Relación de Recurrencia

Para calcular $DP[i][j]$, comparamos los caracteres actuales x_i e y_j :

- Si coinciden ($x_i == y_j$):** Este carácter forma parte de la solución óptima. Sumamos 1 al resultado de resolver el subproblema sin estos dos caracteres:

$$DP[i][j] = DP[i - 1][j - 1] + 1$$

- Si no coinciden ($x_i \neq y_j$):** La SCML óptima vendrá de prescindir de x_i o de prescindir de y_j . Tomamos el máximo de ambas opciones:

$$DP[i][j] = \max(DP[i - 1][j], DP[i][j - 1])$$

Casos Base e Inicialización

Si cualquiera de las dos secuencias está vacía, la longitud de la subsecuencia común es 0:

- $DP[0][j] = 0 \quad \forall j \in [0, n]$
- $DP[i][0] = 0 \quad \forall i \in [0, m]$

2. Algoritmo

El algoritmo consta de dos fases: rellenar la matriz (para obtener la longitud máxima) y reconstruir la solución (para obtener la secuencia de caracteres resultante).

Pseudocódigo

Entrada: $X[1..m]$, $Y[1..n]$

```

// 1. Inicializar la tabla DP con ceros
DP[0..m][0..n] = 0

// 2. Rellenar la tabla por filas o columnas
Para i desde 1 hasta m:
    Para j desde 1 hasta n:
        Si (X[i] == Y[j]):
            DP[i][j] = DP[i-1][j-1] + 1
        Sino:
            DP[i][j] = max(DP[i-1][j], DP[i][j-1])

Longitud_Maxima = DP[m][n]

// 3. Reconstrucción del camino óptimo (Backtracking)
Z = lista vacía
i = m, j = n
Mientras (i > 0 y j > 0):
    Si (X[i] == Y[j]):
        Añadir X[i] al principio de Z
        i = i - 1
        j = j - 1
    Sino si (DP[i-1][j] >= DP[i][j-1]):
        i = i - 1
    Sino:
        j = j - 1

Retornar Longitud_Maxima, Z

```

3. Análisis de Eficiencia

- **Temporal:**
 - **Fase de llenado:** $O(m \cdot n)$, ya que hay dos bucles anidados que recorren la matriz de tamaño $(m + 1) \times (n + 1)$, realizando operaciones de coste constante $O(1)$ en cada celda.
 - **Reconstrucción:** $O(m + n)$, puesto que en cada paso del bucle decrementamos al menos uno de los dos índices (i o j).
 - **Coste Temporal Total:** $O(m \cdot n)$.
- **Espacial:** $O(m \cdot n)$ para almacenar la matriz DP . (Nota para nota extra: Si solo nos pidieran la longitud y no la secuencia, se podría optimizar a $O(\min(m, n))$ guardando únicamente las últimas dos filas de la matriz).